数据结构研究内容：

（1） 同类数据元素间的依赖关系

（2） 针对这些依赖关系的基本操作

第2章 线性表

2.1 线性表的逻辑结构

2.2 线性表的顺序表示和实现

2.3 线性表的链式表示和实现

线性表的基本操作如何表示？ **(见教材P19)**

- 初始化、撤销、清空、判空；
- 求表长、表头、表尾、前趋、后继；
- 读元素、查找（含定位）、遍历；
- 插入、删除

```
InitList( &L );              //建空表，初始化
DestoryList( &L );           //撤销表，释放内存
int LengthList( L );         //求表中元素个数，即表长
POSITION LocateElem (L,ElemType e，compare() ) //查找e
PriorElem( L, cur_e, &pre_e ); //求当前元素e的前驱
NextElem( L, cur_e, &next_e ); //求当前元素e的后继
ListInsertBefore(&L, i, e );      //把e插入到第i个元素之前
ListDelete( &L, i,&e );           //删除第i个元素并“看”此元素
ListTraverse( L, Visit() );       // “看”表中全部元素（遍历）
```

顺序表操作举例

例1（华为招聘试题）： 试用C或类C语言编写一高效算法，将一顺序存储的线性表（设元素均为整型量）中所有零元素向表尾集中，其他元素则顺序向表头方向集中。

初始序列：**5　3　0　0　2　0　7　1**

处理后的序列：**5　3　2　7　1　0　0　0**

算法思路：

- 从前往后扫描，见到0元素则将后续非0元素前移，并继续扫描，全部判别并向前移完后，再把后续部分清零。

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
    {
        num[j]=0;
    }
}
```

j=0

5 3 0 0 2 0 7 1

i=0

Count=0

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
    {
        num[j]=0;
    }
}
```

j=1

5 3 0 0 2 0 7 1

i=0

Count=0

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
      {
        num[j]=0;
      }
}
```

j=1

5 3 0 0 2 0 7 1

i=1

Count=0

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
    {
        num[j]=0;
    }
}
```

j=2

5 3 0 0 2 0 7 1

i=1

Count=0

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
    {
        num[j]=0;
    }
}
```

j=2

5 3 0 0 2 0 7 1

i=2

Count=0

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
    {
        num[j]=0;
    }
}
```

j=2

5 3 0 0 2 0 7 1

i=2

Count=1

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
    {
        num[j]=0;
    }
}
```

j=2

5 3 0 0 2 0 7 1

i=3

Count=1

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
    {
        num[j]=0;
    }
}
```

j=2

5 3 0 0 2 0 7 1

i=3

Count=2

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
    {
        num[j]=0;
    }
}
```

j=2

5 3 0 0 2 0 7 1

i=4

Count=2

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
      {
        num[j]=0;
      }
}
```

j=3

5 3 2 0 2 0 7 1

i=4

Count=2

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
    {                                   j=3
      num[j]=0;     5   3   2   0   2   0   7   1
    }                                   i=5
}
                    Count=2
```

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
      {
        num[j]=0;
      }
}
```

j=3

5 3 2 0 2 0 7 1

i=5

Count=3

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
    {                                   j=3
      num[j]=0;     5   3   2   0   2   0   7   1
    }                                           i=6
}
                    Count=3
```

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
      {
        num[j]=0;
      }
}
```

j=4

5 3 2 7 2 0 7 1

i=6

Count=3

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
      {
        num[j]=0;
      }
}
```

j=4

5 3 2 7 2 0 7 1

i=7

Count=3

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
      {                                          j=5
        num[j]=0;     5   3   2   7   1   0   7   1
      }                                          i=7
}
                      Count=3
```

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
      {
        num[j]=0;
      }
}
```

j=5

5 3 2 7 1 0 7 1

i=8

Count=3

04级曹翼:

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
      {                                                    j=6
        num[j]=0;      5   3   2   7   1   0   0   1
      }                                                    i=8
}
                       Count=3
```

还能不能改进？

04级曹翼：

```
Void sort(int num[],int n)
{       int counter=0,i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
                else {counter++};
        }
        for( j=n-counter; j<n;j++)
      {
        num[j]=0;
      }
}
```

结束

j=7

5 3 2 7 1 0 0 0

i=8

Count=3

```
Void sort(int num[],int n)
{       int i,j=0;
        for (i=0;i<n;i++)
        {
                if (num[i])
                        {num[j++]=num[i]};
        }
        for(; j<n;j++)
        {
                num[j]=0;
        }
}
```

j=5

5 3 2 7 1 0 7 1

i=7

例2　教材例2-1：求两个线性表的“并”，即：

LA U LB = ?

算法至少有两种：

① LA和LB都是无序表，则从LB中取元素逐一与LA中所有元素比较，相同则不再插入LA中；

② 若LA和LB已经是有序表，则“归并”的时间效率可以大大提高。

进一步讨论：

顺序表的存储结构是一维数组，如果插入的元素个数超过数组定义的长度怎么办？

——采用动态分配的一维数组

动态数组简介

先为顺序表空间设定一个初始分配量，一旦因插入元素而空间不足时，可为顺序表增加一个约定长度的空间增量。 存储结构描述如下（见教材P22）：

```
#define LIST_INIT_SIZE      100 //存储空间的初始分配量
#define LISTINCREMENT       10//存储空间的分配增量
Typedef struct{
        ElemType *elem;     //表基址(用指针*elem表示)
        int      length;    //表长度（表中有多少个元素）
        int      listsize;  //当前分配的表尺寸（字节单位）
}SqList;
```

注：三个分量可简写为：L.elem L.length L.listsize

动态创建一个空顺序表的算法：

```
Status InitList_Sq( SqList  &L )     //创建一个空线性表
{   L.elem=(ElemType*)malloc(LIST_INIT_SIZE *
      sizeof(ElemType));
      if(!L.elem)  exit(OVERFLOW); //分配失败，结束程序
       L.length=0;                          //暂无元素放入，空表
       L.listsize=LIST_INIT_SIZE;      //表尺寸=初始分配量
       Return OK;
} //InitList_Sq
```

sizeof(x)算符的意思是：计算变量x的长度(字节数)

malloc (m)函数的意思是：新开一片大小为m字节的连续空间，并把该区首址作为函数值。

动态顺序表的插入算法

```
Status ListInsert_Sq(SqList &L, int i, ElemType e)
{ //在顺序线性表中第i个位置之前插入新的元素e
if( i < 1 or i > L.length+1) return ERROR;  // 检验i 值的合法性
if ( L.length ≥ L.listsize )  //若表长超过表尺寸则要增加尺寸
{  newbase = ( ElemType* ) realloc ( L.elem , (L.listsize +
                LISTINCREMENT )* sizeof ( ElemType ) );
if (newbase=NULL )exit( OVERFLOW ) ; //分配失败则退出并报错
    L.elem = newbase ;                    //重置新基址
    L.listsize = L.listsize + LISTINCREMENT ; } //增加表尺寸
```

realloc (*p, newsize)函数的意思是：新开一片大小为newsize的连续空间，并把以*p为首址的原空间数据都拷贝进去。

```
q = &L.elem[i-1] ;        // q为插入位置。这是没有头结点的情况
for ( p = L.elem[L.length-1] ;  p>=q ; --p ) *(p+1) = *p ;
                    //插入位置及之后的元素统统后移, p为元素位置
*q= e ;                    //插入e
++L.length ;               //增加1个数据元素，则表长+1
 return OK ;
} //ListInsert_Sq
```

动态数组的核心是realloc(void *ptr, newsize)函数！

sizeof(x) 和malloc(m)详见本次PPT后面的解释。

自学：动态顺序表的删除算法

```
Status ListDelete_Sq(SqList &L, int i, ElemType &e)
{   //在顺序表L中删除第 i 个元素，用 e 返回其值

if (i < 1 or  i > L.length) return ERROR;   // i 值不合法，返回

    p=&L.elem[i-1];        // p 是被删除元素的位置
    e=*p;                  //被删除元素的值赋给 e

 q=L.elem+L.length-1;    // q 是表尾的位置
 for(++p; p<=q; p++) *(p-1) = *p;  //待删元素后面的统统前移

   --L.length;             //表长 - 1

     return OK;
}  //ListDelete_Sq
```

2.2节小结

线性表顺序存储结构

特点： 逻辑关系上相邻的两个元素在物理存储位置上也相邻；

优点： 可以随机存取表中任一元素，方便快捷；O(1)

缺点： 在插入或删除某一元素时，需移动大量元素O(n)

解决问题的思路： 改用另一种线性存储方式：

链式存储结构

2.3 线性表的链式表示和实现

2.3.1 链表的表示

2.3.2 链表的实现

2.3.3 链表的运算效率分析

2.3.1 链表的表示

（1）链式存储结构特点：

其结点在存储器中的位置是随意的，即逻辑上相邻的数据元素在物理上不一定相邻。

如何实现？　　通过指针来实现！

让每个存储结点都包含两部分：数据域和指针域

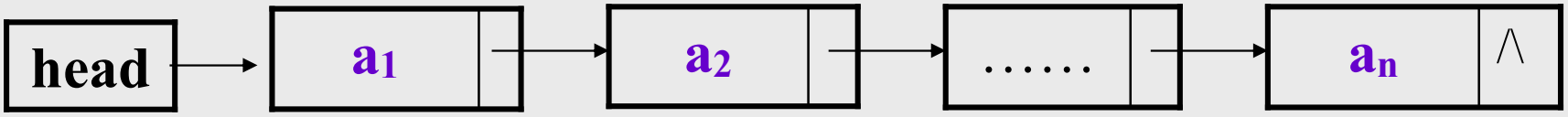
样式：

数据域：存储元素数值数据

指针域：存储直接后继或者直接前驱的存储位置

设计思想：牺牲空间效率换取时间效率

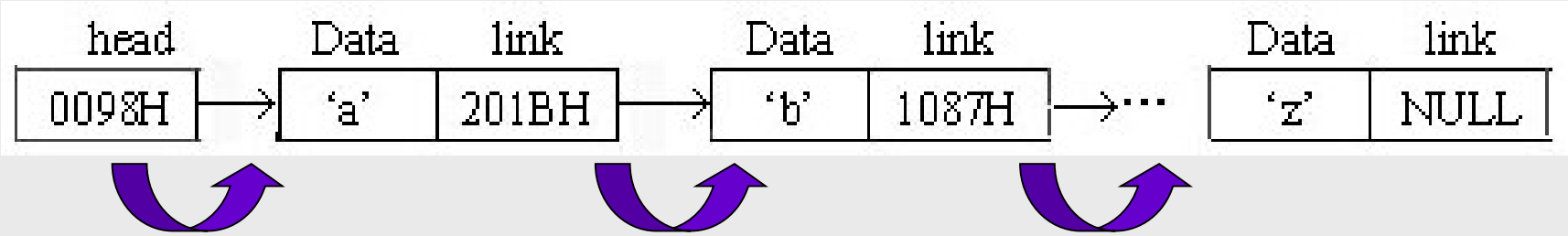
链表存放示意图如下：



例：请画出26 个英文字母表的链式存储结构。

解：该字母表的逻辑结构为：（a, b, … ,y, z)

该字母表在内存中链式存放的样式举例如下：



讨论1：每个存储结点都包含两部分：数据域和__指针域(链域)__。

讨论2：在单链表中，除了首元结点外，任一结点的存储位置由__其直接前驱结点的链域的值__指示。

(2) 与链式存储有关的术语：

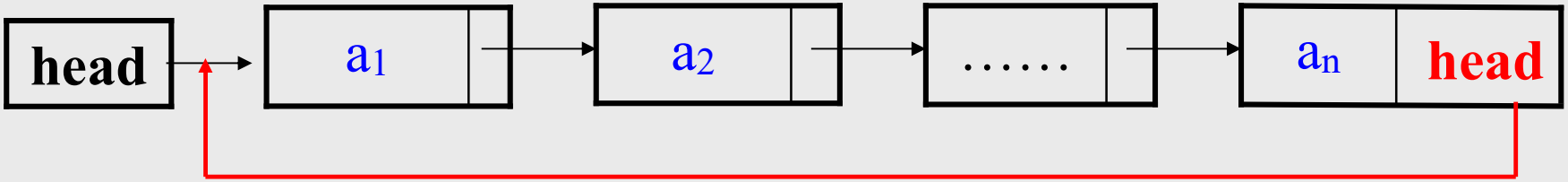
1）结点：数据元素的存储映像。由数据域和指针域两部分组成；

2）链表： n个结点由指针链组成一个链表。它是线性表的链式存储映像，称为线性表的链式存储结构。

3）单链表、双链表、多链表、循环链表：

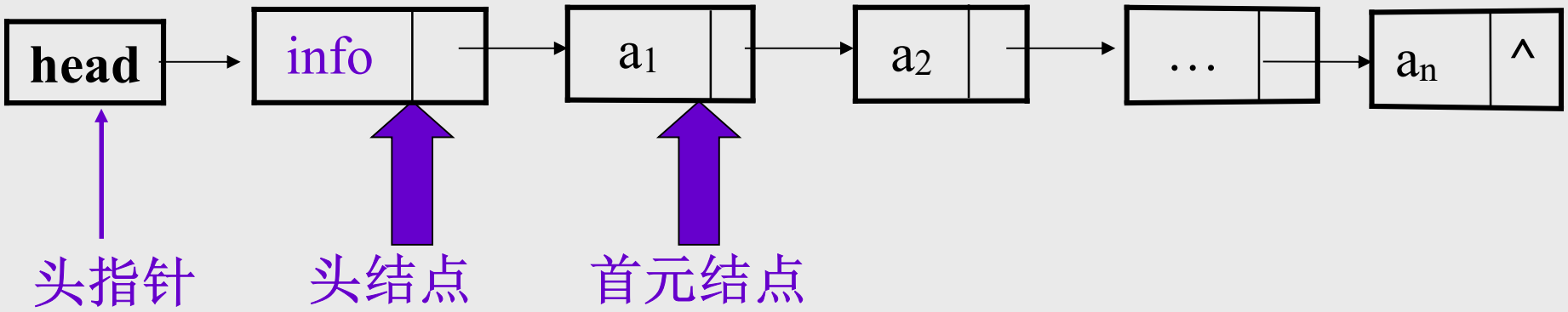
- 结点只有一个指针域的链表，称为单链表或线性链表；
- 有两个指针域的链表，称为双链表（但未必是双向链表）；
- 有多个指针域的链表，称为多链表；
- 首尾相接的链表称为循环链表。

循环链表示意图：



4）头指针、头结点和首元结点的区别

示意图如下：



头指针是指向链表中第一个结点（或为头结点、或为首元结点）的指针；

头结点是在链表的首元结点之前附设的一个结点；数据域内只放空表标志和表长等信息，它不计入表长度。

首元结点是指链表中存储线性表第一个数据元素a_1的结点。

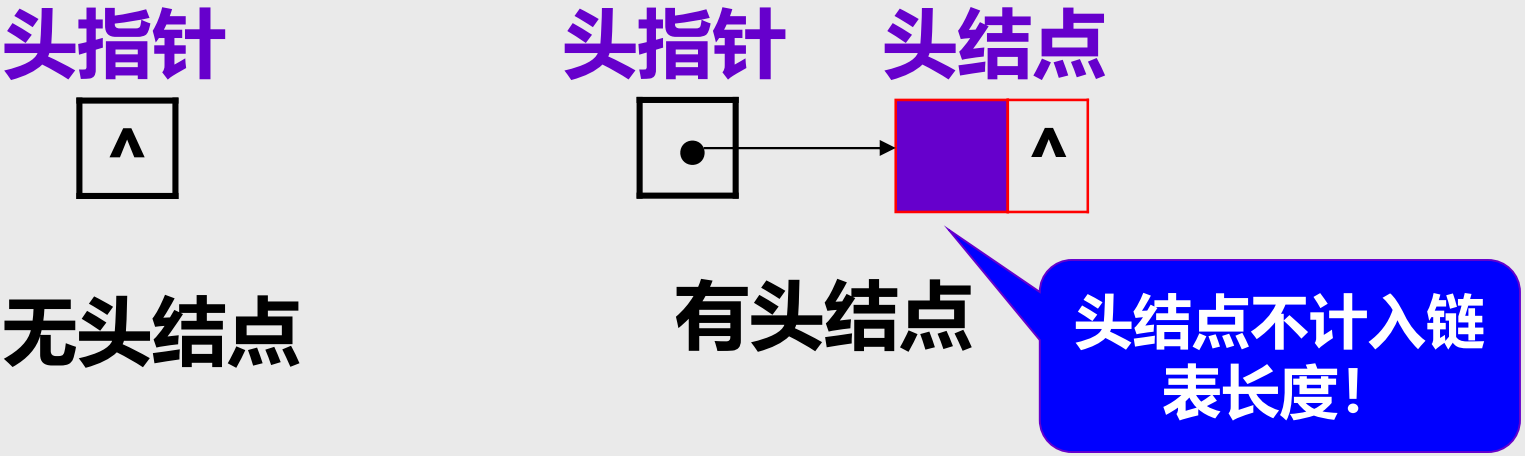
讨论1. 在链表中设置头结点有什么好处?

答：头结点即在链表的首元结点之前附设的一个结点，该结点的数据域可以为空，也可存放表长度等附加信息，其作用是为了对链表进行操作时，可以对空表、非空表的情况以及对首元结点进行统一处理，编程更方便。

讨论2. 如何表示空表?

答：无头结点时，当头指针的值为空时表示空表；

有头结点时，当头结点的指针域为空时表示空表。



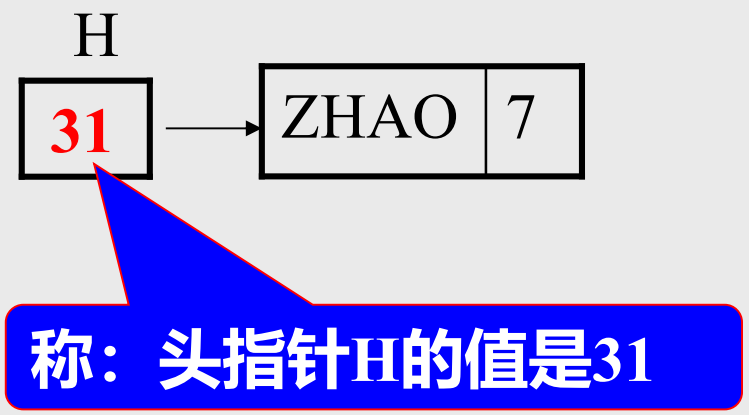
（3）举例

例1：一个线性表的逻辑结构为：

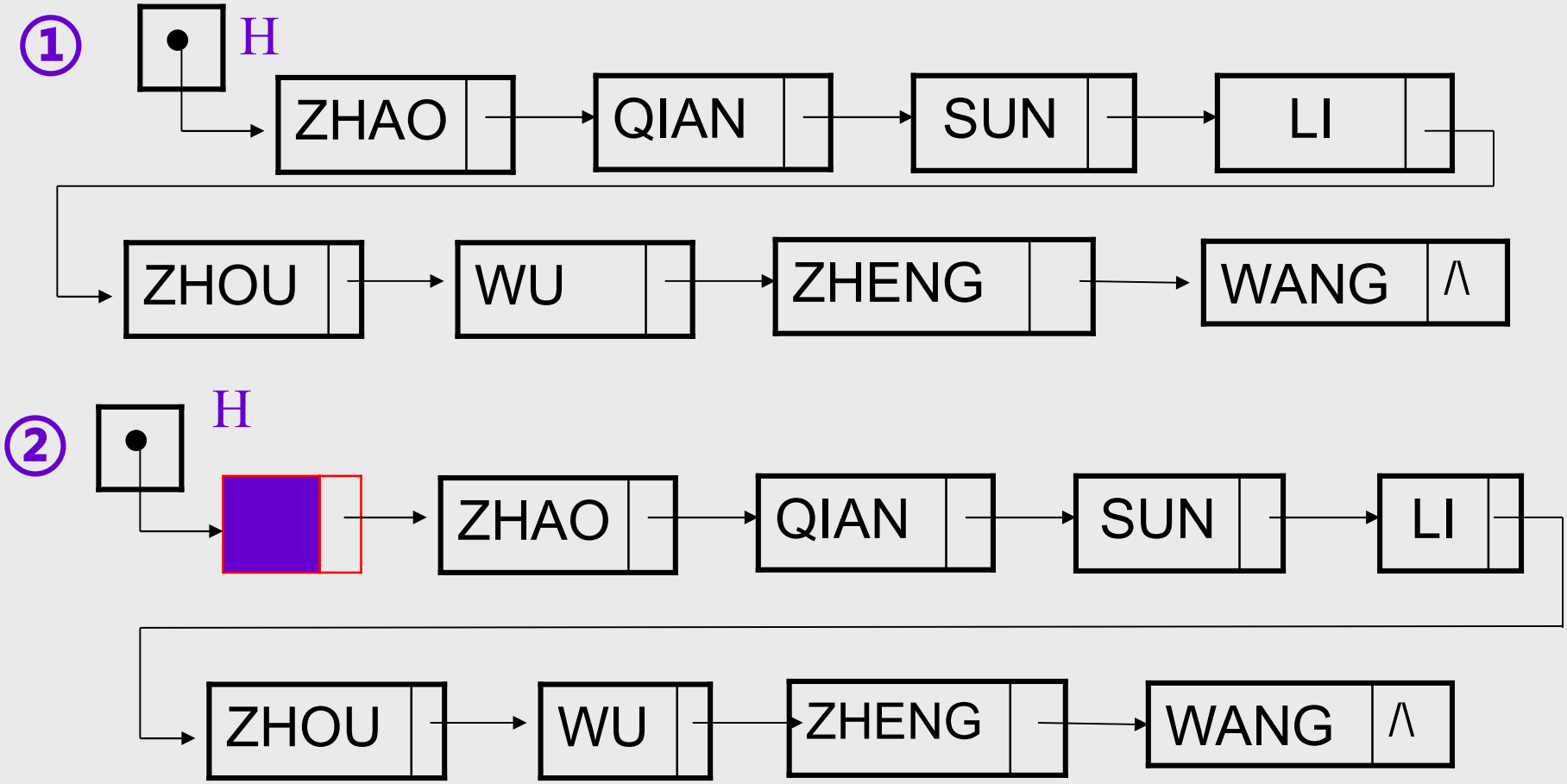
(ZHAO,QIAN,SUN,LI,ZHOU,WU,ZHENG,WANG)，其存储结构用单链表表示如下，请问其头指针的值是多少?

存储地址	数据域	指针域
1	LI	43
7	QIAN	13
13	SUN	1
19	WANG	NULL
25	WU	37
31	ZHAO	7
37	ZHENG	19
43	ZHOU	25

答：**头指针是指向链表中第一个结点的指针，因此关键是要寻找第一个结点的地址。**



上例链表的逻辑结构示意图有以下两种形式：



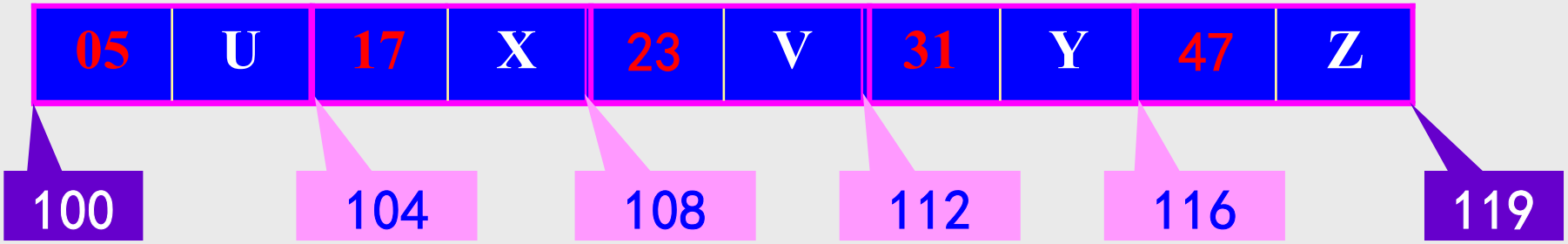
区别：① 无头结点　② 有头结点

头结点不计入链表长度！

例2：

线性表具有两种存储方式，即顺序方式和链接方式。现有一个具有五个元素的线性表L={23，17，47，05，31}，若它以链接方式存储在下列100～119号地址空间中，每个结点由数据（占2个字节）和指针（占2个字节）组成，如下图所示。

其中指针X，Y，Z的值分别为多少？该线性表的首结点起始地址为多少？末结点的起始地址为多少？



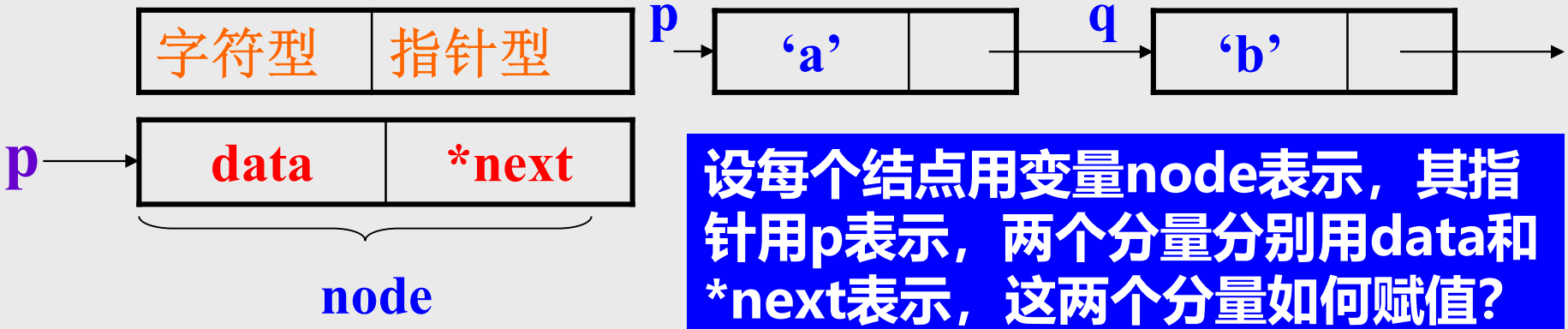
答：X= 116　　Y= NULL/0　　Z= 100，

首址= 108　　末址= 112。

讨论：**链表的数据元素有两个域，不再是简单数据类型，编程时该如何表示?**

答：**因每个结点至少有两个分量，且数据类型通常不一致，所以要采用结构数据类型。**

以26个字母的链表为例，每个结点都有两个分量：



设每个结点用变量node表示，其指针用p表示，两个分量分别用data和*next表示，这两个分量如何赋值?

方式1：　直接表示为　node.data＝'a'；node.next=q

方式2：p指向结点首地址，然后　p->data='a';　p->next=q;

方式3：p指向结点首地址，然后 (*p).data='a'；(*p).next＝q

练习：

设p为指向链表的第i个元素的指针,则第i个元素的数据域写为 p->data 指针域写为 p->next。

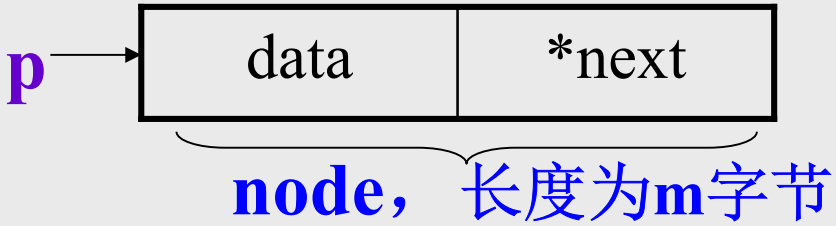
a_i的值　　a_{i+1}的地址

附1：介绍C的三个有用的库函数/算符（都在<stdlib.h> 中）：

sizeof(x)——计算变量x的长度（字节数）；

malloc(m) —开辟m字节长度的地址空间，并返回这段空间的首地址；

free(p) ——释放指针p所指变量的存储空间，即彻底删除一个变量。



sizeof(x)——计算x的长度
malloc(m) —开m字节空间
free(p) ——删除一个变量

问1：自定义结构类型node的长度m是多少？

m＝sizeof(node)　　//单位是字节

问2：结构变量node的首地址(指针p）是多少？

p＝(node*)malloc(m)

```
P->data='a'; p->next=q
```

问3：怎样删除结构变量node？

free(p)　　//只能借助node的指针删除！

附2： 补充结构数据类型的C表示法

① 类型定义和变量说明可以合写为：

```
typedef struct wanger      // wanger是自定义结构类型名称
{  char data;              //定义数据域的变量名及其类型
   struct wanger *next;    //定义指针域的变量名及其类型
}node，  *pointer;         //node是wanger结构类型的类型替代
                           // *pointer是指针型的wanger结构类型的替代
```

② 对于指向结构类型的指针变量，可说明为：

node *p, *q;

或用 struct wanger *p , *q;

注：上面已经定义了node为用户自定义的wanger类型。

单链表的抽象数据类型描述如下（参见教材P28）：

```
Typedef struct Lnode {
    ElemType        data;      //数据域
    struct Lnode   *next;      //指针域
}Lnode, *LinkList;             // *LinkList为Lnode类型的指针
```

至此应可看懂教材P22关于顺序表动态分配的存储结构。其特点是：用结构类型和指针来表示顺序结构，更灵活。

如何具体编程来建立和访问链表？

——链表的实现

单链表如何实现——建表和输出

例：用单链表结构来存放26个英文字母组成的线性表（a，b，c，…，z）,请写出C语言程序。

实现思路：**先开辟头指针，然后陆续为每个结点开辟存储空间并及时赋值，后继结点的地址要提前送给前面的指针。**

先挖“坑”,后种“萝卜”